

Tarea 2: Ejecución de playbooks

Introducción

En esta práctica he configurado mi máquina virtual Debian (`nodo1` , IP `192.168.122.101`) desde `Sergio-PC` mediante el playbook `ansible/ejercicio1/site.yaml` . Partí del repositorio del profesor y trabajé sobre mi fork, llamado `Tarea2-Playbooks` .

1. Fork y directorio de trabajo

Primero hice un **fork** de `josedom24/ejercicios_pi` en GitHub y lo renombré a `Tarea2-Playbooks` . Después, desde mi ordenador, descargué mi fork:

```
git clone git@github.com:Sergio852/Tarea2-Playbooks.git
```

`git clone` copia el fork a mi ordenador; no crea el fork. Entré en el ejercicio que pide el enunciado:

```
cd ~/Documentos/ProyectoIntermodular/Ansible/Tarea2-Playbooks/ansible/ejercicio1
```

Este es el directorio que utilicé durante toda la práctica. `ansible/ejercicio2` también venía en el repositorio, pero **no corresponde a esta entrega**.

2. Inventario y configuración

Abrí `hosts` para configurar el nodo. Los datos de conexión utilizados fueron:

```
nodo1:
  ansible_ssh_host: 192.168.122.101
  ansible_ssh_user: ansible
  ansible_ssh_private_key_file: /home/sergio/.ssh/id_rsa
  ansible_python_interpreter: /usr/bin/python3
```

Estas líneas van bajo el grupo y el nodo que ya existían en `hosts` , respetando su sangría; **no son el fichero entero**. `id_rsa` es la clave privada con la que Ansible se conecta; `id_rsa.pub` es la pública. La línea del intérprete evita el aviso de detección automática de Python.

En el `ansible.cfg` de **este ejercicio** configuré:

```
[defaults]
inventory = hosts
host_key_checking = False
```

Así Ansible utiliza el inventario `hosts` del directorio actual. Comprobé la comunicación con la VM:

```
ansible all -m ping
```

La respuesta esperada es `pong`. Esto comprueba la conexión SSH y que Ansible puede ejecutar código Python en el nodo.

3. Variables y Gathering Facts

Consulté `hosts` para ver las **variables del nodo**: IP, usuario SSH, clave privada e intérprete Python. Consulté `group_vars/all` para ver las **variables de grupo**, aplicables a todos los nodos: `bd_name`, `bd_user` y `bd_pass`. En el fichero del ejercicio, `bd_name` vale `wordpress_bd` y `bd_user` vale `wordpress_user`. No reproduzco la contraseña.

Obtuve las variables detectadas directamente en la VM con:

```
ansible all -m setup
```

`setup` recopila las *facts*. El playbook también muestra `Gathering Facts` al empezar; la plantilla usa, entre otras, `ansible_hostname` y `ansible_distribution`.

4. Completar `site.yaml`

Abrí el playbook original:

```
nano site.yaml
```

No añadí cuatro tareas nuevas: **las cuatro primeras ya estaban**. `hosts: all` hace que se apliquen a los nodos del inventario y `become: true` permite usar `sudo` en la VM. Completé estos huecos:

1. **Actualizar el sistema:** ya venía como `apt: update_cache=yes upgrade=yes`; no tuve que añadir parámetros.
2. **Instalar paquetes:** añadí `git` y `apache2` bajo `loop: .` El `name: "{{ item }}"` existente utiliza cada nombre de la lista.

3. **Copiar un fichero:** conservé `src: files/foo.conf` e indiqué `dest: /etc/foo.conf`.
4. **Publicar la plantilla:** conservé `src: templates/index.j2` e indiqué `dest: /var/www/html/index.html`.

Las partes que tuve que completar en el fichero quedaron así:

```
loop:
  - git
  - apache2
```

```
copy:
  src: files/foo.conf
  dest: /etc/foo.conf
  owner: root
  group: root
  mode: '0644'
```

```
template:
  src: templates/index.j2
  dest: /var/www/html/index.html
  owner: www-data
  group: www-data
  mode: 0644
```

`site.yaml` incluía después dos tareas para crear una base de datos y su usuario con `community.mysql`. **Ya venían en el repositorio**; no las añadí para esta práctica.

5. Completar `templates/index.j2`

Abrí la plantilla:

```
nano templates/index.j2
```

Solo sustituí los tres `modifica_el_nombre`: `bd_name` para el nombre de la base de datos, `bd_user` para su usuario y `ansible_ssh_host` para la IP del nodo. El contenido resultante fue:

```
<html lang="es">
<head>
  <meta charset="utf-8">
  <title>Prueba Ansible</title>
</head>

<body>
  <h1>Gathering Facts</h1>
  <p>Este ordenador se llama: {{ ansible_hostname }}</p>
  <p>S0: {{ansible_distribution}} {{ansible_distribution_release}} </p>
  <h1>Variables declaradas por el usuario a nivel de grupo</h1>
  <p>Nombre de la bd: {{ bd_name }}</p>
  <p>Usuario de la bd: {{ bd_user }}</p>
  <h1>Variables declaradas por el usuario a nivel de nodo</h1>
  <p>IP: {{ ansible_ssh_host }}</p>
</body>
</html>
```

template sustituye las expresiones `{{ ... }}` por sus valores antes de guardar `index.html` en el servidor. Mostrar `wordpress_bd` en la página **no significa que la base de datos ya exista**: solo demuestra que se ha leído la variable.

6. Primera ejecución e incidencias

Desde `ansible/ejercicio1` ejecuté:

```
ansible-playbook site.yaml
```

En el primer intento, `index.j2` tenía una llave mal escrita y apareció `Syntax error in template: unexpected '}'`. La corregí y volví a ejecutar el mismo comando. Entonces la copia de `foo.conf` y la generación de la página funcionaron, pero falló `Crear base de datos` porque faltaba un conector Python para MySQL.

Instalé el conector **en el nodo remoto**:

```
ansible nodo1 -m apt -a "name=python3-pymysql state=present" --become
```

Al repetir el playbook, el error pasó a ser `Conexión rehusada`, porque no había un servidor MySQL/MariaDB disponible. Instalé MariaDB en la VM:

```
ansible nodo1 -m apt -a "name=mariadb-server state=present" --become
```

Después, la tarea ya alcanzaba MariaDB, pero mostró `Access denied for user 'root'@'localhost'`. La opción tratada en el PDF de clase para conectar localmente es añadir `login_unix_socket: /var/run/mysqld/mysqld.sock` dentro de **cada una** de las dos tareas `community.mysql` del playbook. Indica al módulo que se conecte por el socket local, con `become: true`, en lugar de intentar la conexión que estaba fallando.

Importante para la entrega: las ejecuciones que compartí en la conversación todavía mostraban `failed=1`; no tengo aquí una salida posterior que confirme que el playbook completo acabó sin errores. Antes de pegar la captura del punto 2 debo ejecutar de nuevo `ansible-playbook site.yaml` y comprobar que `PLAY RECAP` indica `failed=0` y `unreachable=0`. Si el error persiste, no debo presentar la salida como correcta.

Captura 1: ejecución completa con `failed=0` y `unreachable=0`, una vez obtenida.

7. Segunda ejecución: idempotencia

Ejecuté otra vez:

```
ansible-playbook site.yaml
```

En la salida que guardé, las tareas de actualización, paquetes, copia y plantilla aparecieron en `ok` y el resumen mostró `changed=0`: Ansible las comprobó, pero no volvió a realizar cambios porque ya estaban en el estado deseado. Esta propiedad se llama **idempotencia**. Aquella salida terminaba todavía con `failed=1` por la tarea de base de datos, así que para entregar una segunda ejecución completa sin errores tendré que repetirla tras resolver ese fallo.

Captura 2: segunda ejecución completa; explicar `ok` frente a `changed`.

8. Borrar y restaurar `foo.conf`

Para probar la recuperación borré **el fichero del servidor**, no el original `files/foo.conf` de mi repositorio:

```
ansible nodo1 -m file -a "path=/etc/foo.conf state=absent" --become
```

La salida mostró `state: absent` y `changed: true`. Volví a lanzar:

```
ansible-playbook site.yaml
```

La tarea `Copiar fichero a la máquina remota` apareció como `changed: [nodo1]`: detectó que faltaba `/etc/foo.conf` y volvió a copiarlo. Las tareas que ya estaban bien permanecieron

en `ok` . Esa ejecución aún terminaba con el fallo posterior de base de datos, pero **la restauración de `foo.conf` sí funcionó.**

Captura 3: borrado remoto y nueva ejecución con la tarea de copia en `changed` .

9. Comprobación en el navegador

Abrí `http://192.168.122.101` en el navegador del nodo de control. Se mostró el `index.html` generado por la plantilla, con `sergiotarea1` , `Debian trixie` , `wordpress_bd` , `wordpress_user` y `192.168.122.101` . Esto comprueba la publicación de la página y el uso de *facts*, variables de grupo y variable de nodo; no prueba por sí solo la creación de la base de datos.

Captura 4: navegador con la URL y la página visibles.

10. Ficheros y fork para entregar

<https://github.com/Sergio852/Tarea2-Playbooks>